Ir Além 1 - Comunicação REST e e-mail simulado

CardioIA Conectada - Fase 3, Capítulo 1
Responsável acadêmico: Guilherme Yamada Dantas - RM rm568506
Data de referência: 30/10/2026

Objetivo e fluxo implementado

O módulo demonstra comunicação HTTP entre um cliente Python e uma API local, processamento determinístico de sinais sintéticos e criação automática de uma mensagem de alerta. O percurso executado é: amostra sintética > POST /samples > validação > regras > transação SQLite > arquivo MIME .eml > GET da amostra. Todo o módulo usa a biblioteca padrão Python, permitindo reprodução sem pacotes externos. O cliente usa urllib.request; o servidor usa http.server, ligado apenas a 127.0.0.1. O Cap02 fundamenta a separação cliente/servidor, verbos HTTP, JSON e tratamento de status; a escolha de stdlib reduz dependências neste experimento.

Cada amostra contém identificador, marcador synthetic=true, timestamp ISO 8601 com fuso, bpm e temperatura em graus Celsius. O servidor rejeita campos ausentes ou adicionais, IDs inadequados, valores não finitos, booleanos usados como números ou valores fora dos intervalos do experimento. O corpo tem limite de 4096 bytes. A criação retorna 201, a consulta 200, recursos inexistentes 404 e dados inválidos 400. Tipo de mídia diferente de JSON retorna 415; tamanho inválido retorna 413. GET /samples apresenta os 100 registros mais recentes e informa essa limitação.

Regras de alerta e simulação

As regras escolhidas são bpm >120 e temperatura >38 °C. Valores exatamente 120 e 38 não disparam alerta. Uma ou ambas as condições podem ser registradas no campo alerts. Os limiares são exclusivamente didáticos, sem interpretação clínica. DHT22 mede temperatura ambiente: não se pode concluir febre de sua leitura. O módulo não implementa ausência de movimento porque não recebe essa grandeza.

Uma amostra em alerta gera automaticamente uma mensagem com assunto identificado como SIMULACAO, cabeçalho X-CardioIA-Simulated=true e corpo contendo a amostra e os motivos. Os endereços pertencem ao domínio reservado example.invalid. O arquivo é salvo na pasta outbox; não há SMTP, credenciais, entrega ou contato com terceiros. Uma amostra normal não produz arquivo. Assim, a funcionalidade demonstrada é a automação da preparação local do e-mail, e não entrega por um provedor.

Persistência, repetição e falhas

SQLite preserva as amostras e o conteúdo MIME. Um sample_id já recebido com o mesmo conteúdo retorna 200 sem duplicar a mensagem; conteúdo divergente produz 409. O registro e o conteúdo do e-mail são gravados na mesma transação. Após o commit, o arquivo é materializado por escrita temporária e substituição. Na próxima inicialização ou repetição, uma mensagem faltante é reconstituída a partir do banco. Falhas locais de persistência retornam 500 e o cliente pode repetir o mesmo ID. Isso não protege contra perda do disco; também não estabelece entrega exactly-once em serviços de e-mail, pois nenhum serviço externo faz parte desta implementação.

Evidências executadas e reprodução

Foram executados sete testes de integração com sockets HTTP reais em loopback, portas efêmeras e diretórios temporários. Todos passaram. As verificações cobrem POST/GET/listagem, amostra normal, alertas individuais e simultâneos, limites exatos, conteúdo MIME, idempotência e conflito, entradas inválidas, NaN, mídia/tamanho, rotas inexistentes, restrição do cliente ao loopback e persistência após reinício com recuperação de arquivo. O log está em docs/rest/test-results.txt. A execução de rest/run_evidence.py também preservou respostas POST/GET, banco e .eml na pasta datada docs/rest/evidence/. São evidências reais locais, sem alegação de nuvem.

Para reproduzir: executar python3 rest/server.py e, em outro terminal, python3 rest/client.py. Para os testes: python3 -m unittest discover -s rest -p 'test_*.py' -v. A alternativa python3 rest/run_evidence.py inicia e encerra seu próprio servidor, criando uma nova pasta de evidências para preservar o histórico das execuções.

Limitações e referência

O servidor é um protótipo sequencial, sem autenticação, TLS, fila distribuída ou retenção automática. A flag synthetic é uma declaração do produtor, não prova de anonimização. O módulo não deve receber dados pessoais nem ser exposto à rede. A ligação com MQTT não foi implementada neste Ir Além: o produtor testado é o cliente REST Python. O Cap02 e seus exemplos foram consultados; os originais foram preservados e nenhuma rotina de envio real foi executada. Referência: FIAP, Fase 3, Cap02, REST e E-mail como Motores de Processos em RPA, seções 3.1 e 4 (2025).